

Project #2. OpenSSD Firmware Hacking and Block-Level FTL Implementation

OBJECTIVE

In this project, you will explore how the Cosmos+ OpenSSD [2] firmware manages flash memory through its Flash Translation Layer (FTL). Building on your understanding from the previous phase, you will analyze the firmware's internal I/O translation flow and transform the existing *Page-Level FTL* into a *Block-Level FTL* implementation.

Recall that page-level mapping provides fine-grained flexibility but consumes more on-board memory for mapping tables, whereas block-level mapping reduces metadata overhead at the cost of coarser granularity. By the end of this assignment, you will design and implement a simplified block-level mapping scheme inside the Cosmos+ firmware and verify that host I/O requests can complete without errors using the evaluation tool.

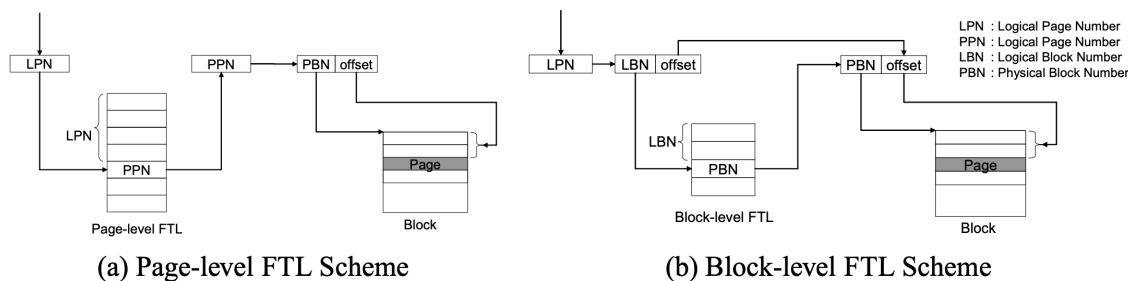


FIGURE 1. Page-level vs. Block-level FTL schemes [1].

Duration. May 12 (Tue) – May 26 (Tue) (2 Weeks)

TASK DESCRIPTION

Task 1 (5/12): Understanding Internal I/O Flow in Cosmos+ OpenSSD Firmware.

- Analyze how the Cosmos+ firmware processes host I/O requests and translates logical addresses to NAND physical locations.
- Trace the full I/O flow across the request transform, address translation, data buffering, and garbage collection routines.
- Focus on understanding how the page-level mapping works at the code level before transitioning to block-level logic.

Checklist.

- **No submission is required** for this task.

Task 2 (5/14): Implementing a Block-Level FTL.

- Extend the existing *Page-Level FTL* to support **Block-Level mapping granularity**.
- The logical-to-physical translation should now operate at the NAND flash block level rather than per-NAND page.
- You must make the minimal modification required to achieve block-level behavior while ensuring that the provided evaluation program completes successfully without runtime aborts.

Checklist.

- **You can modify only two files:**
 - `address_translation.(c|h)`

All other firmware files must remain unmodified.

- Your implementation should allow the host evaluation program (`ftl_bench`) to:
 - Start and finish without device aborts or firmware assertion failures in both `seq` and `rand` modes.
 - Print valid mapping summary and space utilization logs on the device side.
 - Report `result: OK=... FAIL=... elapsed time=...` on the host side.
- You are **not** required to implement a perfectly correct or optimal Block-Level FTL. Your priority is to ensure a working firmware execution path and to understand the trade-offs of coarser-grained mapping.
- In `seq` mode, `FAIL=0` is required. In `rand` mode, data mismatches may occur; *explain the cause in your report*.

MATERIALS PROVIDED

- **Host-side evaluation program** (`prj2-host-evaluation/`): Includes `Makefile`, `ftl_bench.cc`, `nvme_passthru.(h|cc)`, and `README.md`. Running this binary verifies your firmware by issuing read/write workloads. Please reload the OpenSSD firmware before each benchmark run to ensure a clean FTL state.
- **Workspace patch directory** (`prj2-workspace-patch/`): Includes minimal code patches for `nvme.h` and `nvme_io_cmd.c` to support device-side mapping summary and space utilization log reports.

STUDENT SUBMISSION

Submit the following items under the naming convention below:

- 1) **Modified firmware source**: (`prj2-ftlpatch-{studentID}.tar.gz`) — must include only your `address_translation.(c|h)` file(s).
- 2) **Project Report (PDF)**: (`prj2-document-{studentID}.pdf`)

Submission Window. Opens: May 12 (Tue) 15:00 Closes: May 26 (Tue) 23:59

EVALUATION CRITERIA

- **Functionality (60%)** — The firmware must (1) execute and complete the host-side benchmark without any firmware errors or hangs, and (2) correctly perform address translation at the **NAND block level** instead of the page level.
- **Report Quality (40%)** — clarity, explanation of design, limitations, and improvement plans.

NOTES

- Use administrative privileges (`sudo`) only when absolutely necessary, and always double-check before executing commands.
- Handle the Cosmos+ OpenSSD board with care. Physical mishandling can permanently damage the hardware.
- Any destructive damage to the board or the Linux development machine — whether physical or software-induced — is entirely your responsibility. Such incidents may delay your progress, but all delays and losses during project execution must be borne by you alone.

REFERENCES

- [1] Aayush Gupta, Youngjae Kim, and Bhuvan Uргаonkar. Dftl: a flash translation layer employing demand-based selective caching of page-level address mappings. In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, ASPLOS, page 229–240, New York, NY, USA, 2009.
- [2] Jaewook Kwak, Sangjin Lee, Kibin Park, Jinwoo Jeong, and Yong Ho Song. Cosmos+ openssd: Rapid prototype for flash storage systems. *ACM Trans. Storage*, 16(3), July 2020.